

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2022/2023

SEPTEMBER EXAMINATION

**UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND
ARCHITECTURE**

SATURDAY, 24 SEPTEMBER 2022

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING
BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING

Instruction to Candidates:

This question paper consists of **TWO (2)** sections.

SECTION A consists of **THREE (3)** questions. **Answer ALL questions.**

SECTION B consists of **TWO (2)** questions. Answer **ONLY ONE (1)** question. Should a candidate answer more than **ONE (1)** questions in **SECTION B**, marks will only be awarded for the **FIRST** question in the order the candidate submits the answers.

Each question carries 25 marks.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

SECTION A (Answer ALL questions)

Q1. (a) Answer the following questions. Show all your workings.

- (i) Convert 513_8 to decimal and BCD. (3 marks)
- (ii) Solve the arithmetic operation $0101\ 0010_{\text{Gray}} - 68_{10}$ using 2's complement method. Give your final answer in binary and indicate whether overflow happens. Justify your answer. Assuming an 8-bits system. (8 marks)

(b) (i) Simplify the following logic function using Boolean Algebra:

$$f(a, b, c) = ac' + b'c' + ac + b'c + a'b \quad (4 \text{ marks})$$

- (ii) Derive and simplify the logic function Y in Figure 1.1. Then, draw the schematic diagram of the simplified logic function. (10 marks)

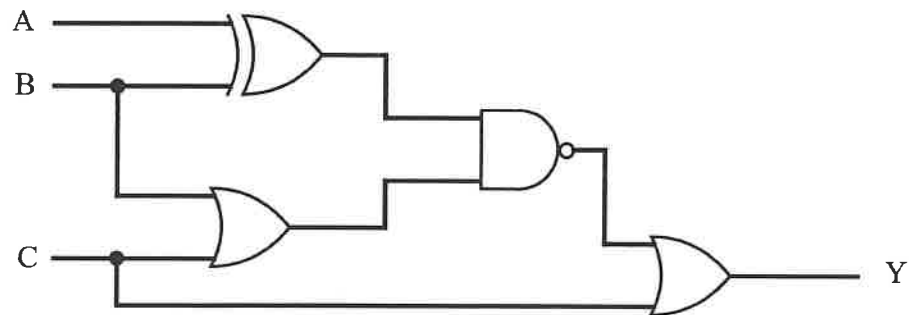


Figure 1.1

[Total : 25 marks]

Q2. (a) Consider the following machine codes:

$0x016C5020 \dots (1)$

$0xAD0A0008 \dots (2)$

- (i) By referring to the appendices, translate the machine code above into MIPS assembly language. Show all your workings. (10 marks)
- (ii) State the addressing mode of machine codes (1) and (2). (2 marks)

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Q2. (Continued)

(b) Consider the MIPS assembly code below.

```
sub  $s0, $t1, $t0 ... (1)
mult $s0, $t2      ... (2)
mflo $s1           ... (3)
addi $v0, $s1, 0x72 ... (4)
addi $sp, $sp, 8   ... (5)
jr   $ra           ... (6)
```

- (i) Assume that the registers \$t0, \$t1 and \$t2 hold the decimal values of 40, 55 and 7, respectively. State the values of \$s0, \$s1 and \$v0. (5 marks)
- (ii) By referring to the appendices, translate the assembly code (6) into machine code. Show all your workings. (4 marks)
- (iii) Briefly explain the role of registers \$v0 and \$sp. (4 marks)

[Total : 25 marks]

Q3. (a) Modern processor typically utilizes pipeline instead of single-cycle microarchitecture.

- (i) Compare the datapath of single-cycle and pipeline processor by giving THREE (3) differences between the two. (6 marks)
- (ii) With the help of an appropriate diagram, explain the impact of using pipeline architecture compared to single-cycle in terms of throughput and latency. (8 marks)

- (b) (i) Give TWO (2) advantages of dynamically linked libraries (DLLs) over statically linked library. (3 marks)
- (ii) Compare compiler and interpreter by providing TWO (2) advantages and TWO (2) disadvantages for each translator as shown in Table 3.1. (8 marks)

	Advantages	Disadvantages
Compiler	1. 2.	1. 2.
Interpreter	1. 2.	1. 2.

Table 3.1

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

SECTION B (Choose only ONE question)

- Q4. (a) I/O devices is one of the most complex areas in computer design.
- (i) List ONE (1) reason why managing I/O devices is challenging. (2 marks)
 - (ii) There are two main methods that a CPU can accessed to I/O devices' addresses. List and elaborate these TWO (2) methods. (6 marks)
- (b) Describe THREE (3) advantages of USB compared to other legacy varieties of serial and parallel port. (6 marks)
- (c) Typically, a processor needs to perform a set of actions when an exception or interrupt occurs.
- (i) Give TWO (2) examples of exception or interrupt. (4 marks)
 - (ii) Briefly describe the interrupt processing in MIPS processor. (7 marks)
- [Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Q5. (a) The memory capacity of a computer can be visualized by a hierarchy of components.

- (i) Draw the memory hierarchy and compare TWO (2) of their important features. You should include at least THREE (3) components in the memory hierarchy. (5 marks)
- (ii) List and explain the TWO (2) types of localities to make memory accesses fast. (6 marks)

(b) Figure 5.1 illustrates the datapath abstract implementation view of a processor.

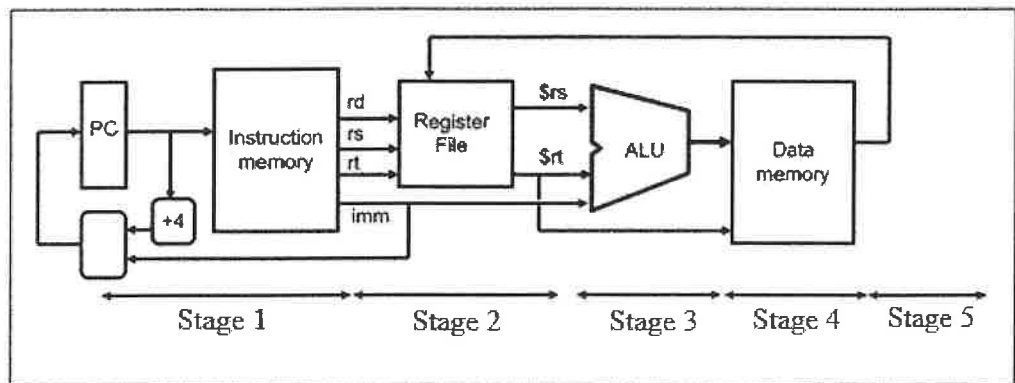


Figure 5.1

- (i) Elaborate the activities done in each of the stages illustrated in Figure 5.1 when executing the instruction *lw \$rt, imm(\$rs)*. (10 marks)
 - (ii) Name any TWO (2) information that need to be decoded in stage 2. (4 marks)
- [Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix A: MIPS Reference Data

MIPS Reference Data

①



CORE INSTRUCTION SET

CORE INSTRUCTION SET				OPCODE / FUNCTION (Hex)
NAME, MNEMONIC	FOR- MAT	OPERATION (in Verilog)		
Add	add	R R[rd] = R[rs] + R[rt]	(1)	0 / 20 _{hex}
Add Immediate	addi	I R[rt] = R[rs] + SignExtImm	(1,2)	8 _{hex}
Add Imm. Unsigned	addiu	I R[rt] = R[rs] + SignExtImm	(2)	9 _{hex}
Add Unsigned	addu	R R[rd] = R[rs] + R[rt]		0 / 21 _{hex}
And	and	R R[rd] = R[rs] & R[rt]		0 / 24 _{hex}
And Immediate	andi	I R[rt] = R[rs] & ZeroExtImm	(3)	c _{hex}
Branch On Equal	beq	I if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4 _{hex}
Branch On Not Equal	bne	I if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5 _{hex}
Jump	j	J PC=JumpAddr	(5)	2 _{hex}
Jump And Link	jal	J R[31]=PC+8;PC=JumpAddr	(5)	3 _{hex}
Jump Register	jr	R PC=R[rs]		0 / 08 _{hex}
Load Byte Unsigned	lbu	I R[rt]={24'b0,M[R[rs] +SignExtImm](7:0)}	(2)	24 _{hex}
Load Halfword Unsigned	lhu	I R[rt]={16'b0,M[R[rs] +SignExtImm](15:0)}	(2)	25 _{hex}
Load Linked	ll	I R[rt] = M[R[rs]+SignExtImm]	(2,7)	30 _{hex}
Load Upper Imm.	lui	I R[rt] = {imm, 16'b0}		f _{hex}
Load Word	lw	I R[rt] = M[R[rs]+SignExtImm]	(2)	23 _{hex}
Nor	nor	R R[rd] = ~(R[rs] R[rt])		0 / 27 _{hex}
Or	or	R R[rd] = R[rs] R[rt]		0 / 25 _{hex}
Or Immediate	ori	I R[rt] = R[rs] ZeroExtImm	(3)	d _{hex}
Set Less Than	slt	R R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / 2a _{hex}
Set Less Than Imm.	slti	I R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a _{hex}
Set Less Than Imm. Unsigned	sltiu	I R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b _{hex}
Set Less Than Unsig.	sltu	R R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / 2b _{hex}
Shift Left Logical	sll	R R[rd] = R[rt] << shamt		0 / 00 _{hex}
Shift Right Logical	srl	R R[rd] = R[rt] >> shamt		0 / 02 _{hex}
Store Byte	sb	I M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28 _{hex}
Store Conditional	sc	I M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0	(2,7)	38 _{hex}
Store Halfword	sh	I M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29 _{hex}
Store Word	sw	I M[R[rs]+SignExtImm] = R[rt]	(2)	2b _{hex}
Subtract	sub	R R[rd] = R[rs] - R[rt]	(1)	0 / 22 _{hex}
Subtract Unsigned	subu	R R[rd] = R[rs] - R[rt]		0 / 23 _{hex}

- (1) May cause overflow exception
- (2) SignExtImm = { 16{immediate[15]}, immediate }
- (3) ZeroExtImm = { 16{1b'0'}, immediate }
- (4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
- (5) JumpAddr = { PC+4{31:28}, address, 2'b0 }
- (6) Operands containing unsigned numbers (vs. 2's comp.)
- (7) Atomic test&set pair: R[r] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

R	opcode		rs	rt	rd	shamt	funct
	31	26 23	31 20	16 15	11 10	6 5	0
I	opcode		rs	rt	immediate		
	31	26 25	21 20	16 15	0		
J	opcode		address				
	31	26 25	0				

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design*, 4th ed.

ARITHMETIC CORE INSTRUCTION SET

② OPCODE

NAME, MNEMONIC	FOR-MAT	OPERATION	/ FUNCT (Hcx)
Branch On FP True	bc1.t	FI if(FPcond)PC=PC+4+BranchAddr (4)	11/8/1/--
Branch On FP False	bc1.f	FI if(!FPcond)PC=PC+4+BranchAddr(4)	11/8/0/--
Divide	div.r	R Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	0/--/--/1a
Divide Unsigned	divu.r	R Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6) 0/--/--/1b
FP Add Single	add.s	FR F[fd]=F[fs]+F[ft]	11/10/--/0
FP Add Double	add.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}	11/11/--/0
FP Compare Single	c.x.s*	FR FPcond = (F[fs] op F[ft]) ? 1 : 0	11/10/--/y
FP Compare Double	c.x.d*	FR FPcond = ({F[fs],F[fs+1]} op {F[ft],F[ft+1]}) ? 1 : 0	11/11/--/y
* (x is eq, lt, or le) (op is ==, <, or <=) (y is 32, 3c, or 3e)			
FP Divide Single	div.s	FR F[fd]=F[fs]/F[ft]	11/10/--/3
FP Divide Double	div.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}	11/11/--/3
FP Multiply Single	mul.s	FR F[fd]=F[fs]*F[ft]	11/10/--/2
FP Multiply Double	mul.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}	11/11/--/2
FP Subtract Single	sub.s	FR F[fd]=F[fs]-F[ft]	11/10/--/1
FP Subtract Double	sub.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}	11/11/--/1
Load FP Single	lwc1	I F[rt]=M[R[rs]+SignExtImm]	(2) 31/--/--/--
Load FP Double	ldc1	I F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2) 35/--/--/--
Move From Hi	mfhi	R R[rd]=Hi	0/--/--/10
Move From Lo	mflo	R R[rd]=Lo	0/--/--/12
Move From Control	mfc0	R R[rd]=CR[rs]	10/0/--/0
Multiply	mult	R {Hi,Lo} = R[rs]*R[rt]	0/--/--/18
Multiply Unsigned	multu	R {Hi,Lo} = R[rs]*R[rt]	(6) 0/--/--/19
Shift Right Arith.	sra	R R[rd]=R[rt]>>>shamt	0/--/--/3
Store FP Single	swc1	I M[R[rs]+SignExtImm]=F[rt]	(2) 39/--/--/--
Store FP Double	sdc1	I M[R[rs]+SignExtImm]=F[rt]; M[R[rs]+SignExtImm+4]=F[rt+1]	(2) 3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	fnt	ft	fs	fd	funct
	31 26 25	21 20	16 15	11 10	6 5	0
FI	opcode	fnt	fl	immediate		
	31 26 25	21 20	16 15	0		

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix A: MIPS Reference Data (continued)

③

OPCODES, BASE CONVERSION, ASCII SYMBOLS

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexadecimal	ASCII Character	Decimal	Hexadecimal	ASCII Character
(1)	sll	addf	00 0000	0	0	NUL	64	40	@
		subf	00 0001	1	1	SOH	65	41	A
j	srl	mulf	00 0010	2	2	STX	66	42	B
jal	sra	divf	00 0011	3	3	ETX	67	43	C
beq	sllv	sqrtof	00 0100	4	4	EOT	68	44	D
bne		absof	00 0101	5	5	ENQ	69	45	E
blez	srlv	movf	00 0110	6	6	ACK	70	46	F
bgtz	sra	negf	00 0111	7	7	BEL	71	47	G
addi	jr		00 1000	8	8	BS	72	48	H
addiu	jalr		00 1001	9	9	HT	73	49	I
slli	movz		00 1010	10	a	LF	74	4a	J
slliu	movn		00 1011	11	b	VT	75	4b	K
andi	syscall	round.wf	00 1100	12	c	FF	76	4c	L
ori	break	trunc.wf	00 1101	13	d	CR	77	4d	M
xori		ceil.wf	00 1110	14	e	SO	78	4e	N
lui	sync	floor.wf	00 1111	15	f	SI	79	4f	O
(2)									
	mthi		01 0000	16	10	DLE	80	50	P
	mtbi		01 0001	17	11	DC1	81	51	Q
	mflo	movzf	01 0010	18	12	DC2	82	52	R
	mtlo	movnf	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	]
			01 1110	30	1e	RS	94	5e	^
			01 1111	31	1f	US	95	5f	_
lb	add	cvt.s.f	10 0000	32	20	Space	96	60	`
lh	addu	cvt.d.f	10 0001	33	21	!	97	61	a
lwl	sub		10 0010	34	22	"	98	62	b
lwr	subu		10 0011	35	23	#	99	63	c
lbu	and	cvt.w.f	10 0100	36	24	\$	100	64	d
lhu	or		10 0101	37	25	%	101	65	e
lwr	xor		10 0110	38	26	&	102	66	f
	nor		10 0111	39	27	'	103	67	g
sb			10 1000	40	28	(	104	68	h
sh			10 1001	41	29	)	105	69	i
swl	sllt		10 1010	42	2a	*	106	6a	j
sw	slltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
			10 1110	46	2e	.	110	6e	n
swr			10 1111	47	2f	/	111	6f	o
cache									
ll	tge	c.f.f	11 0000	48	30	0	112	70	p
lwc1	tgeu	c.un.f	11 0001	49	31	1	113	71	q
lwc2	tlbt	c.eq.f	11 0010	50	32	2	114	72	r
pref	tlbtu	c.ueq.f	11 0011	51	33	3	115	73	s
	teq	c.o.lt.f	11 0100	52	34	4	116	74	t
ldc1		c.o.lt.f	11 0101	53	35	5	117	75	u
ldc2	tne	c.o.le.f	11 0110	54	36	6	118	76	v
		c.o.le.f	11 0111	55	37	7	119	77	w
sc		c.s.f.f	11 1000	56	38	8	120	78	x
swc1		c.ngle.f	11 1001	57	39	9	121	79	y
swc2		c.seq.f	11 1010	58	3a	:	122	7a	z
		c.ngl.f	11 1011	59	3b	;	123	7b	{
			11 1100	60	3c	<	124	7c	
sdc1		c.ngle.f	11 1101	61	3d	=	125	7d	}
sdc2		c.le.f	11 1110	62	3e	>	126	7e	~
		c.ngt.f	11 1111	63	3f	?	127	7f	DEL

(1) opcode(31:26) = 0

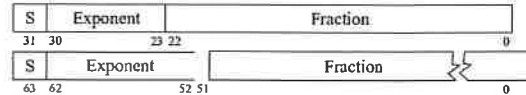
(2) opcode(31:26) = 17_{ten} (11_{hex}); if fmt(25:21) = 16_{ten} (10_{hex}) f = s (single);
if fmt(25:21) = 17_{ten} (11_{hex}) f = d (double)

IEEE 754 FLOATING-POINT STANDARD

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

where Single Precision Bias = 127,
Double Precision Bias = 1023.

IEEE Single Precision and Double Precision Formats:

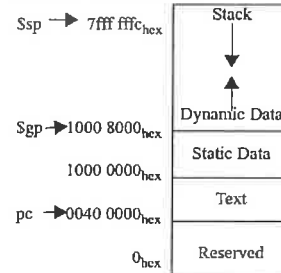


IEEE 754 Symbols

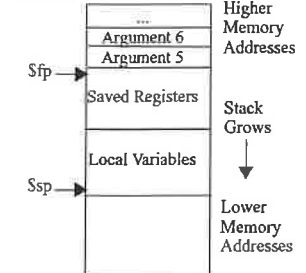
Exponent	Fraction	Object
0	0	± 0
0	≠ 0	± Denorm
1 to MAX - 1	anything	± Fl. Pt. Num.
MAX	0	±∞
MAX	≠ 0	NaN

S.P. MAX = 255, D.P. MAX = 2047

MEMORY ALLOCATION



STACK FRAME

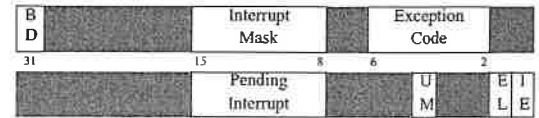


DATA ALIGNMENT

Double Word							
Word				Word			
Halfword		Halfword		Halfword		Halfword	
Byte	Byte	Byte	Byte	Byte	Byte	Byte	Byte
0	1	2	3	4	5	6	7

Value of three least significant bits of byte address (Big Endian)

EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS



BD = Branch Delay, UM = User Mode, EL = Exception Level, IE = Interrupt Enable

EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RI	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10³ for Disk, Communication; 2³ for Memory)

SIZE	PRE-FIX	SIZE	PRE-FIX	SIZE	PRE-FIX	SIZE	PRE-FIX
10 ³ , 2 ¹⁰	Kilo-	10 ¹⁵ , 2 ⁵⁰	Peta-	10 ⁻³	milli-	10 ⁻¹⁵	femto-
10 ⁶ , 2 ²⁰	Mega-	10 ¹⁸ , 2 ⁶⁰	Exa-	10 ⁻⁶	micro-	10 ⁻¹⁸	atto-
10 ⁹ , 2 ³⁰	Giga-	10 ²¹ , 2 ⁷⁰	Zetta-	10 ⁻⁹	nano-	10 ⁻²¹	zepto-
10 ¹² , 2 ⁴⁰	Tera-	10 ²⁴ , 2 ⁸⁰	Yotta-	10 ⁻¹²	pico-	10 ⁻²⁴	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix B: MIPS Instruction Syntax

Instruction	Syntax	Operation
add	add rd, rs, rt	$[rd] = [rs] + [rt]$
add immediate	addi rt, rs, imm	$[rt] = [rs] + \text{SignImm}$
add immediate unsigned	addiu rt, rs, imm	$[rt] = [rs] + \text{SignImm}$
add unsigned	addu rd, rs, rt	$[rd] = [rs] + [rt]$
and	and rd, rs, rt	$[rd] = [rs] \& [rt]$
and immediate	andi rt, rs, imm	$[rt] = [rs] \& \text{ZeroImm}$
branch if equal	beq rs, rt, label	if $([rs] == [rt])$ PC = BTA
branch if not equal	bne rs, rt, label	if $([rs] != [rt])$ PC = BTA
divide	div rs, rt	$[lo] = [rs]/[rt], [hi] = [rs]\%[rt]$
divide unsigned	divu rs, rt	$[lo] = [rs]/[rt], [hi] = [rs]\%[rt]$
jump	j label	PC = JTA
jump and link	jal label	\$ra = PC + 4, PC = JTA
jump register	jr rs	PC = [rs]
load byte	lb rt, imm(rs)	$[rt] = \text{SignExt}([Address]7:0)$
load byte unsigned	lbu rt, imm(rs)	$[rt] = \text{ZeroExt}([Address]7:0)$
load upper immediate	lui rt, imm	$[rt] = \{imm, 16'b0\}$
load word	lw rt, imm(rs)	$[rt] = [Address]$
move from hi	mfhi rd	$[rd] = [hi]$
move from lo	mflo rd	$[rd] = [lo]$
move from/to coprocessor 0	mfc0 rt, rd /mtc0 rt, rd	$[rt] = [rd]/[rd] = [rt]$
move to hi	mthi rs	$[hi] = [rs]$
move to lo	mtlo rs	$[lo] = [rs]$
multiply	mult rs, rt	$\{[hi], [lo]\} = [rs] \times [rt]$
multiply (32-bit result)	mul rd, rs, rt	$[rd] = [rs] \times [rt]$
multiply unsigned	multu rs, rt	$\{[hi], [lo]\} = [rs] \times [rt]$
nor	nor rd, rs, rt	$[rd] = \sim([rs] \mid [rt])$
or	or rd, rs, rt	$[rd] = [rs] \mid [rt]$
or immediate	ori rt, rs, imm	$[rt] = [rs] \mid \text{ZeroImm}$
set less than	slt rd, rs, rt	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$
set less than immediate	slti rt, rs, imm	$[rs] < \text{SignImm} ? [rt] = 1 : [rt] = 0$
set less than immediate unsigned	sltiu rt, rs, imm	$[rs] < \text{SignImm} ? [rt] = 1 : [rt] = 0$
set less than unsigned	sltu rd, rs, rt	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$
shift left logical	sll rd, rt, shamt	$[rd] = [rt] \ll \text{shamt}$
shift right arithmetic	sra rd, rt, shamt	$[rd] = [rt] \ggg \text{shamt}$
shift right logical	srl rd, rt, shamt	$[rd] = [rt] \gg \text{shamt}$
store byte	sb rt, imm(rs)	$[Address]7:0 = [rt]7:0$
store word	sw rt, imm(rs)	$[Address] = [rt]$
subtract	sub rd, rs, rt	$[rd] = [rs] - [rt]$
subtract unsigned	subu rd, rs, rt	$[rd] = [rs] - [rt]$
xor	xor rd, rs, rt	$[rd] = [rs] \wedge [rt]$
xor immediate	xori rt, rs, imm	$[rt] = [rs] \wedge \text{ZeroImm}$